

PINN volatility study — what to improve next, item by item

4 October 2026 · Baseline: runs/main (commit 7dcead9): IV surface 7.29, IV ATM 23.37, GARCH 33.44, LSTM 36.21 RMSE (PINN Mode P)

How to ask

- Pick one or more item IDs below and tell me, e.g. "**do A2**", "**do Phase 2**", "**do A1 and A2 without Mode H**", or "**do A4 with $\lambda = 0.05$ and 0.5** ". You can change any default in the item when you ask ("do A3 but keep quarterly refits").
- For each item I will: implement the change on branch `claude/pinn-research-pipeline-4u075w`, add tests, run pytest, run the experiment on the laptop in the background, and send you the key numbers against the baseline plus anything that went wrong.
- Every experiment writes to its **own folder** `runs/<ID>/` with its own config `configs/<ID>.yaml`, so `runs/main` stays as the untouched baseline and results are always comparable.
- I commit locally after each item; I **push only when you say so**. `dataset/` and `data-clean/` are never modified.
- Where an item needs a decision from you (marked **Decision needed**), I ask before starting.

Ground rules for every item

Rule	Value
Baseline	runs/main (commit 7dcead9): IV surface 7.29, IV ATM 23.37, GARCH 33.44, LSTM 36.21 RMSE (PINN Mode P)
Test sample	unchanged: 1,993,333 monthly SPX options, 2020-01-02 → 2023-12-29, unless the item says otherwise
Metrics	RMSE / MAE / MAPE vs market mid and vs analytical Black-Scholes; IV-RMSE; DM tests — always for GARCH, LSTM and IV side by side
No look-ahead	every new input uses only information available at the previous close; a test checks it
Runtime guard	long stages run with <code>nohup</code> + unbuffered logs; if a run projects beyond 8 h I report and reduce seeds/subsample before continuing

Overview

Phase	Aim	Items	Compute
Phase 1	Make experiments cheap and statistics sound	E1, E3, E2, B1, B2, A5, C1	≈ 1 day of coding; evaluate-only runs (minutes)
Phase 2	Fix the comparison itself	A1, A2	≈ 1 h compute (Mode P + BS only)
Phase 3	Give the LSTM a fair chance	A3	≈ 1–2 h compute
Phase 4	The PINN-specific question: data vs physics	A4	≈ 8 h (overnight)
Phase 5	Robustness and solver quality	C2, D1	≈ 6–8 h
Phase 6	Write-up	W1	minutes

ID	Title	Priority	Depends on	Decision
E1	Timestamped logs and resumable pricing	Low (enabler)	—	—
E3	Re-price only what changed	High (enabler)	E1	—
E2	Freeze the environment	Low	—	—
B1	Multiple-testing control and per-regime tests	Medium	—	—
B2	Volatility-unit and vega-weighted errors	Low	—	—
A5	Label the same-day contract IV as an oracle	Medium	—	—
C1	Finish documenting the monthly-dating issue	Low	—	—
A1	Smile-adjusted forecast treatments	High	E3 (recommended)	—
A2	Volatility-risk-premium adjustment	High	E3 (recommended); A1 for the combined treatment	yes
A3	A stronger LSTM	High	E3 (recommended)	yes
A4	Hybrid PINN: λ grid	Medium	E1, E3	yes
C2	Use 2012-2018 options and run the weekly robustness sample	Medium	E1	yes
D1	Tighten the physics-only PINN	Low	E3	—
W1	Final write-up	—	the phases you chose	—

Phase 1 — Make experiments cheap and statistics sound

Compute: $\approx$ 1 day of coding; evaluate-only runs (minutes)

E1 · Timestamped logs and resumable pricing

Priority: Low (enabler) · Compute: coding $\approx$ 2 h; verification $\approx$ 30 min · Depends on: —

Goal	Never lose a long run again and always know how far it is.
Why (evidence)	$\approx$ 50 min were lost to a silent, buffered log; price ($\approx$ 5 h) holds everything in memory and writes only at the end, so a crash at hour 4 loses all of it.
Change	• <code>cli.py</code>: force line-buffered stdout; prefix every log line with a timestamp and elapsed time. • <code>pipeline/pricing.py</code>: save each quarterly block to <code>runs/<ID>/blocks/<quarter>.parquet</code> and skip blocks already on disk (resume). • Store predictions as Parquet with float32 prices ($\approx$ 1.3 GB $\rightarrow$ $\approx$ 0.4 GB).
Config	• <code>price.resume</code>: <code>true</code> (default)
Run	• <code>python -m pinnvol price -c configs/default.yaml</code> — kill it after one block, restart, confirm it resumes.
Done when	• Log lines carry timestamps; a restarted run resumes from the last finished block. • evaluate on the Parquet output reproduces <code>runs/main/report.md</code> numbers exactly.
I report back	Resume test result; file-size change; confirmation that numbers are identical.

E3 · Re-price only what changed

Priority: High (enabler) · Compute: coding $\approx$ 2 h · Depends on: E1

Goal	Let new treatments or engines be priced in minutes instead of re-running the full 5-hour stage.
Why (evidence)	Most improvements add one volatility input or change one engine. Today price recomputes every engine $\times$ treatment, including Mode H (the 5-hour part).
Change	• <code>pipeline/pricing.py</code>: <code>price.only_engines</code> / <code>price.only_treatments</code> filters; merge new columns into an existing predictions file from a <code>price.base_run</code> folder. • <code>pinn.mode_h.treatments</code> already exists — document it.
Config	• <code>price.base_run</code>: <code>runs/main</code> • <code>price.only_engines</code>: <code>[bs, pinn_p]</code> • <code>price.only_treatments</code>: <code>[garch_smile, ...]</code>
Run	• Re-price one existing treatment with the filter and compare with <code>runs/main</code>.
Done when	• Re-pricing a single treatment with BS + Mode P takes $<$ 15 min and matches <code>runs/main</code> to floating-point precision.
I report back	Timing of a single-treatment re-price; equality check.

E2 · Freeze the environment

Priority: Low · Compute: ≈ 30 min · Depends on: —

Goal	Make every number in the thesis reproducible.
Why (evidence)	torch 2.14.1 + CUDA 13 and all other packages were installed fresh during the run; versions are not recorded.
Change	• Add requirements-lock.txt (from <code>uv pip freeze</code>). • Write git_commit, package versions and GPU name into each run folder (run_info.json).
Run	• Any stage; check run_info.json.
Done when	• Each runs/<ID>/ contains config_used.yaml and run_info.json.
I report back	The lock file and an example run_info.json.

B1 · Multiple-testing control and per-regime tests

Priority: Medium · Compute: evaluate-only, ≈ 5 -10 min · Depends on: —

Goal	Report only differences that survive correction for running ≈ 70 tests.
Why (evidence)	DM tests are reported at nominal p-values. GARCH vs LSTM ($p \approx 0.03$) may not survive correction.
Change	• evaluation/stats.py: Holm-Bonferroni adjusted p-values per engine; Model Confidence Set (Hansen, Lunde & Nason 2011) with a stationary block bootstrap (numpy only). • pipeline/report.py: DM tests also per regime (calm / COVID crash / bear 2022) and per maturity bucket; new columns p_holm, table mcs.csv.
Config	• evaluation.mcs: {alpha: 0.10, block: 10, reps: 2000}
Run	• <code>python -m pinnvol evaluate -c configs/default.yaml</code> (no re-pricing).
Done when	• dm_tests.csv has p_holm; mcs.csv lists the surviving set per engine; report.md updated. • Unit test: MCS keeps the true best model on simulated losses.
I report back	Which comparisons stay significant after correction; the MCS per engine and regime.

B2 · Volatility-unit and vega-weighted errors

Priority: Low · Compute: evaluate-only · Depends on: —

Goal	Compare fairly across maturities and with the literature.
Why (evidence)	Price RMSE is dominated by long-dated options (120-180 d errors $\approx 5\times$ the 7-30 d errors).
Change	• evaluation/metrics.py: vega-weighted RMSE; IV-RMSE moved to the headline table; RMSE by maturity bucket in report.md.
Run	• evaluate only.
Done when	• Headline table shows price RMSE, IV-RMSE and vega-weighted RMSE side by side.
I report back	The new headline table.

A5 · Label the same-day contract IV as an oracle

Priority: Medium · Compute: evaluate-only · Depends on: —

Goal	Stop the oracle being read as a competing method.
Why (evidence)	iv_contract_same_day prices an option with its own implied volatility (RMSE 0.19).
Change	• pipeline/report.py: move oracle rows into a separate 'reference bounds' table; exclude them from DM tests and rankings.
Run	• evaluate only.
Done when	• No oracle rows in rankings or DM tables.
I report back	Updated report.md.

C1 · Finish documenting the monthly-dating issue

Priority: Low · Compute: minutes · Depends on: —

Goal	Nobody repeats the 2015 mistake when using the cleaned files directly.
Why (evidence)	The repo README now warns about it (commit 5d72bab), but data-profile/ does not, and data-clean/ must not be edited.
Change	• Add the finding (weekday table, dates) to data-profile/spx_data_profile.md.
Run	• —
Done when	• The data profile has a 'Monthly expiry dating' section.
I report back	Link to the new section.

Phase 2 — Fix the comparison itself

Compute: ≈ 1 h compute (Mode P + BS only)

A1 · Smile-adjusted forecast treatments

Priority: High · Compute: ≈ 30 –60 min (without Mode H); ≈ 2 h with Mode H at 2 seeds · Depends on: E3 (recommended)

Goal	Compare forecasting methods on the volatility level with an identical smile.
Why (evidence)	GARCH/LSTM errors concentrate in the wings (K/F 0.80–0.90: GARCH 37.3 vs IV surface 5.3). Today the comparison partly measures 'smile vs no smile', not forecasting skill.
Change	• vol/IMPLIED.py: from the previous day's fitted smile, the shape factor $s(K,T) = IV_{surf}(K,T) / IV_{ATM}(T)$. • pipeline/volatility.py: new columns $\sigma_{<m>smile} = \sigma_{<m>} \times s(K,T)$ for $m \in \{garch, lstm, har, iv_atm\}$. (iv_atm_smile should reproduce iv_surface — a built-in sanity check.) • Test: no same-day information enters $s(K,T)$.
Config	• treatments: [..., garch_smile, lstm_smile, har_smile, iv_atm_smile] • price.only_engines: [bs, pinn_p] (Mode H optional)
Run	• vol $\rightarrow$ price (filtered, needs E3) $\rightarrow$ evaluate, output runs/A1/.
Done when	• iv_atm_smile RMSE within 0.5 of iv_surface. • Table: raw vs smile-adjusted RMSE for GARCH, LSTM, HAR, IV ATM; moneyness chart redone.
I report back	How much of the GARCH/LSTM vs IV gap remains once all share the smile.

A2 · Volatility-risk-premium adjustment

Priority: High · Compute: $\approx$ 30–60 min · Depends on: E3 (recommended); A1 for the combined treatment

Goal	Separate the level shift between physical and risk-neutral volatility from forecasting skill.
Why (evidence)	All forecast-based inputs underprice: mean bias -19.6 (GARCH), -21.1 (LSTM) index points; implied vol sits above realised vol on average.
Change	• <code>vol/</code>: rolling variance ratio $k_t = (252\text{-day mean of IV_ATM,30d}^2) \div (252\text{-day mean of } \hat{\sigma}_m^2)$, using data up to $t-1$ only. • New columns $\sigma_{<m>_vrp} = \sqrt{k_t} \times \sigma_{<m>}$, and the combination $<m>_vrp_smile$ with A1.
Config	• <code>volatility.vrp</code>: {window: 252, horizon: 30} • <code>treatments</code>: [..., garch_vrp, lstm_vrp, garch_vrp_smile, lstm_vrp_smile]
Run	• <code>vol</code> $\rightarrow$ filtered price $\rightarrow$ evaluate, output runs/A2/.
Done when	• Bias of GARCH/LSTM inputs reported before and after; no-look-ahead test passes.
Decision needed	Whether adjusted treatments become the headline results or a separate robustness table.
I report back	Bias and RMSE before/after; the remaining gap to IV = the forecasting contribution.

Phase 3 — Give the LSTM a fair chance

Compute: $\approx$ 1–2 h compute

A3 · A stronger LSTM

Priority: High · Compute: $\approx$ 15–30 min per variant for vol; pricing $\approx$ 30 min each with E3 · Depends on: E3 (recommended)

Goal	Test whether a well-specified deep-learning forecaster can match GARCH or IV.
Why (evidence)	Best in validation 2019 (QLIKE 0.231), worst in test (0.932); forecasts too low (17.0% vs 19.3% realised at 21 d); COVID-crash pricing RMSE 74.7 vs 37.7 for GARCH. The model is small, return-only and refit quarterly.
Change	• A3a — monthly refit (config only: <code>volatility.lstm.refit: M</code>). • A3b — longer history: SPX closes from 2000 through <code>paths.extra_prices_csv</code> (already supported). • A3c — implied-vol features: VIX from FRED (VIXCLS, free, same downloader as the rates) and the lagged ATM IV term structure. • A3d — target log realised variance at each option's own horizon. • A3e — validation window that includes a stress period (e.g. 2018 Q4).
Config	• <code>volatility.lstm: {refit: M, features: [returns, vix, iv_atm], target: log_rv}</code>
Run	• Ablation: A3a, then +A3b, +A3c, +A3d, each as runs/A3x/, compared on QLIKE first, pricing second.
Done when	• Test QLIKE (21 d) of the best variant $\leq$ GARCH's 0.657, or a documented reason why not. • COVID-crash pricing RMSE reported for every variant.
Decision needed	Source for SPX closes before 2012 (A3b): download from a public source, or you provide a file.
I report back	An ablation table: which change helped and by how much.

Phase 4 — The PINN-specific question: data vs physics

Compute: $\approx$ 8 h (overnight)

A4 · Hybrid PINN: λ grid

Priority: Medium · Compute: $\approx$ 2 h per λ at 2 seeds $\approx$ 8 h total (overnight) · Depends on: E1, E3

Goal	Show how much a PINN should trust market data versus physics.
Why (evidence)	Only $\lambda = 0.1$ was run: small gains (GARCH 33.4 $\rightarrow$ 32.4) and drift from BS of 0.9–2.7 points.
Change	• Config only (<code>pinn.mode_h.lambdas</code> is already a list); a new figure: market RMSE and BS-gap against λ for each volatility input.
Config	• <code>pinn.mode_h: {lambdas: [0.01, 0.1, 1, 10], seeds: [0, 1]}</code>
Run	• price with <code>only_engines: [pinn_h]</code> (needs E3), output runs/A4/.
Done when	• λ curve per treatment; recommended λ with justification.
Decision needed	The λ values (default 0.01, 0.1, 1, 10).
I report back	The λ trade-off figure and table.

Phase 5 — Robustness and solver quality

Compute: $\approx$ 6–8 h

C2 · Use 2012–2018 options and run the weekly robustness sample

Priority: Medium · Compute: $\approx$ 6 h (one full run) · Depends on: E1

Goal	Check that conclusions hold beyond monthly SPX options.
Why (evidence)	After the fix, $\approx$ 0.97 M training-period options exist but no model uses them; only monthlies were tested.
Change	• Use the 2012–2018 panel for VRP calibration (A2) and LSTM-IV history (A3c). • A robustness run with <code>sample.contract_family: all</code> (adds SPXW weeklies, PM-settled).
Config	• <code>configs/C2_spxw.yaml</code>: <code>sample.contract_family: all</code>, <code>sample.max_options_per_day</code> if needed
Run	• Full pipeline into <code>runs/C2/</code> (subsampled if the projection exceeds 8 h).
Done when	• Robustness table: same ranking (or documented differences) on the wider sample.
Decision needed	Subsample size if the SPXW run is too slow.
I report back	Side-by-side headline tables, monthly vs all contracts.

D1 · Tighten the physics-only PINN

Priority: Low · Compute: $\approx$ 10–20 min training per variant + $\approx$ 20 min pricing · Depends on: E3

Goal	Bring the PINN's error vs Black-Scholes below 0.1 index points.
Why (evidence)	Solver error is 0.31–0.37 points ($\approx$ 50 $\times$ Crank-Nicolson); with the oracle IV only 84% of PINN prices fall inside the bid-ask spread vs 99.99% for BS.
Change	• Try, in order: <code>weighting: lra</code>; L-BFGS 1,500 $\rightarrow$ 3,000 steps; width 64 $\rightarrow$ 128; a convexity (no-arbitrage) penalty on $\partial^2 V / \partial K^2 \geq 0$.
Config	• <code>pinn.mode_p</code>: <code>{weighting: lra, lbfgs_steps: 3000}</code>, <code>pinn.hidden: 128</code>
Run	• <code>train-pinn</code> per variant into <code>runs/D1x/</code>; re-price BS + Mode P only (E3).
Done when	• Validation MAE vs closed form and test solver RMSE per variant; best one $<$ 0.1 points.
I report back	Variant table; whether the headline results change (expected: no).

Phase 6 — Write-up

Compute: minutes

W1 · Final write-up

Priority: — · Compute: minutes · Depends on: the phases you chose

Goal	Turn the results into thesis-ready tables and figures.
Why (evidence)	Every phase changes numbers; the report should be regenerated from the final runs.
Change	• Extend <code>reports/build_progress_report.py</code> to compare baseline vs improved runs; export LaTeX tables.
Run	• <code>python reports/build_progress_report.py</code>
Done when	• Updated PDF and LaTeX tables from the final runs.
I report back	The new PDF.

Example requests

- "do Phase 1" — E1, E3, E2, B1, B2, A5, C1 in that order, one report at the end.

- "do A1 and A2" — both treatments in one vol + filtered price run (runs/A1A2/).
- "do A3a and A3c only" — monthly refit plus VIX features, no longer history.
- "do A4 with $\lambda = 0.05, 0.5, 5$ and 3 seeds" — overrides the defaults.
- "do D1 but only LRA" — one variant.